BÁO CÁO TUẦN 1 – JAVA CORE

1. OOP

1.1. Encapsulation

Khái niệm

Encapsulation là việc đóng gói dữ liệu và các phương thức xử lý dữ liệu vào trong một class, đồng thời kiểm soát quyền truy cập vào dữ liệu.

Thông thường:

- private được sử dụng cho dữ liệu.
- public method được cung cấp để truy cập hoặc thay đổi dữ liệu một cách có kiểm soát.

Ví dụ:

```
class Student {
    private String name;
    private int age;

    public void setAge(int age) {
        if (age >= 0 && age <= 100) {
            this.age = age;
        }
    }

    public int getAge() {
        return age;
    }
}
```

Không thể làm:

```
Student s = new Student();

s.age = -100;
```

vì age là private.

Thay vào đó:

s.setAge(-100);

Method setAge() có thể kiểm tra dữ liệu trước khi thay đổi object.

Ý nghĩa

Encapsulation không chỉ nhằm mục đích "ẩn dữ liệu", mà quan trọng hơn là:

Đảm bảo object luôn ở trạng thái hợp lệ và kiểm soát cách dữ liệu bên trong được thay đổi.

Ví dụ một tài khoản ngân hàng không nên cho phép:

account.balance = -1000000;

mà phải thông qua method:

account.withdraw(100000);

để kiểm tra số dư và các điều kiện liên quan.

1.2. Inheritance

Khái niệm

Inheritance cho phép một class con kế thừa thuộc tính và phương thức từ class cha.

Ví dụ:

```
class Animal {

    public void eat() {
        System.out.println("Animal is eating");
    }

    public void sound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
```

```
    @Override
    public void sound() {
        System.out.println("Woof");
    }
}
```

Sử dụng:

```
Dog dog = new Dog();


dog.eat();
dog.sound();
```

Dog sử dụng được eat() từ Animal và override sound().

Khi nào nên sử dụng?

Inheritance phù hợp khi tồn tại quan hệ:

IS-A

Ví dụ:

Dog IS-A Animal

Cat IS-A Animal

Car IS-A Vehicle

Không nên sử dụng inheritance chỉ để tái sử dụng code.

Nếu quan hệ không phải IS-A thì thường nên xem xét **Composition**.

1.3. Polymorphism – Đa hình

Polymorphism cho phép cùng một interface hoặc method nhưng có thể có nhiều implementation khác nhau.

Ví dụ:

```
class Animal {


    public void sound() {
        System.out.println("Animal sound");
    }
```

```
}

class Dog extends Animal {

    @Override
    public void sound() {
        System.out.println("Woof");
    }
}

class Cat extends Animal {

    @Override
    public void sound() {
        System.out.println("Meow");
    }
}
```

Có thể sử dụng:

```
Animal animal1 = new Dog();
Animal animal2 = new Cat();

animal1.sound();
animal2.sound();
```

Kết quả:

```
Woof
Meow
```

Biến có kiểu Animal, nhưng method được thực thi phụ thuộc vào object thực tế.

Đây là **runtime polymorphism**.

Ý nghĩa

Polymorphism giúp code phụ thuộc vào abstraction thay vì implementation.

Ví dụ:

List<String> list = new ArrayList<>();

Code phía trên phụ thuộc vào List thay vì trực tiếp phụ thuộc vào ArrayList.

Sau này có thể thay:

List<String> list = new LinkedList<>();

mà phần lớn code phía trên không cần thay đổi.

1.4. Abstraction

Abstraction là **việc chỉ expose những đặc điểm và hành vi cần thiết, đồng thời ẩn đi các chi tiết triển khai bên trong.**

Ví dụ:

```
abstract class Animal {

    public abstract void sound();

    public void eat() {
        System.out.println("Animal is eating");
    }
}
```

Class con phải implement:

```
class Dog extends Animal {

    @Override
    public void sound() {
        System.out.println("Woof");
    }
}
```

Người sử dụng chỉ cần biết:

```
animal.sound();
```

mà không cần quan tâm bên trong sound() được triển khai như thế nào.

2. SOLID Principles

SOLID gồm 5 nguyên lý:

Chữ	Nguyên lý
S	Single Responsibility Principle
O	Open/Closed Principle
L	Liskov Substitution Principle
I	Interface Segregation Principle
D	Dependency Inversion Principle

Mục tiêu của SOLID là giúp code:

- Dễ maintain.
- Dễ mở rộng.
- Dễ test.
- Giảm coupling.
- Hạn chế thay đổi code không cần thiết.

2.1. S – Single Responsibility Principle

Một class nên chỉ có một responsibility và chỉ có một lý do chính để thay đổi.

Ví dụ không tốt:

```
class UserService {

    public void createUser() {
    }

    public void sendEmail() {
    }

    public void saveToDatabase() {
    }
```

```
}
```

Class này đang làm nhiều việc:

User management

Email

Database

Có thể tách thành:

```
class UserService {
    public void createUser() {
    }
}

class EmailService {
    public void sendEmail() {
    }
}

class UserRepository {
    public void save() {
    }
}
```

Lợi ích

Khi thay đổi logic gửi email, không cần sửa UserService.

Code có responsibility rõ ràng và dễ test hơn.

2.2. O – Open/Closed Principle

Software entities should be open for extension but closed for modification.

Có nghĩa:

- Có thể mở rộng behavior.
- Hạn chế sửa code đã ổn định.

Ví dụ không tốt:

```
class PaymentService {

    public void pay(String type) {

        if (type.equals("MOMO")) {
            // MoMo payment
        } else if (type.equals("VISA")) {
            // Visa payment
        } else if (type.equals("PAYPAL")) {
            // Paypal payment
        }
    }
}
```

Mỗi lần thêm phương thức thanh toán phải sửa PaymentService.

Có thể dùng abstraction:

```
interface Payment {

    void pay();
}
```

Các implementation:

```
class MomoPayment implements Payment {

    @Override
    public void pay() {
        System.out.println("Pay with MoMo");
    }
}

class VisaPayment implements Payment {
```

```
    @Override
    public void pay() {
        System.out.println("Pay with Visa");
    }
}
```

Service:

```
class PaymentService {

    public void processPayment(Payment payment) {
        payment.pay();
    }
}
```

Khi cần thêm:

```
class PaypalPayment implements Payment {

    @Override
    public void pay() {
        System.out.println("Pay with Paypal");
    }
}
```

Không cần sửa PaymentService.

2.3. L – Liskov Substitution Principle

Object của subclass phải có thể thay thế object của superclass mà không làm thay đổi tính đúng đắn của chương trình.

Ví dụ kinh điển:

```
class Bird {
    public void fly() {
    }
```

```
}
```

Nếu tạo:

```
class Penguin extends Bird {
}
```

thì Penguin không thể bay.

Khi đó abstraction Bird có vấn đề vì không phải mọi bird đều fly.

Nên thiết kế:

```
class Bird {
    public void eat() {
    }
}


interface Flyable {
    void fly();
}
```

Chỉ những loài biết bay mới implement:

```
class Eagle extends Bird implements Flyable {


    @Override
    public void fly() {
        System.out.println("Eagle flying");
    }
}
```

Còn:

```
class Penguin extends Bird {
}
```

Ý nghĩa

Không nên tạo inheritance relationship nếu subclass không thể thực sự tuân thủ behavior của superclass.

2.4. I – Interface Segregation Principle

Client không nên bị ép phụ thuộc vào những method mà nó không sử dụng.

Ví dụ không tốt:

```
interface Worker {

    void work();

    void eat();

    void sleep();
}
```

Một class robot:

```
class Robot implements Worker {

    public void work() {
    }

    public void eat() {
        // Robot doesn't eat
    }

    public void sleep() {
        // Robot doesn't sleep
    }
}
```

Interface quá lớn.

Có thể tách:

```
interface Workable {
    void work();
```

```
}

interface Eatable {
    void eat();
}

interface Sleepable {
    void sleep();
}
```

Robot chỉ cần:

```
class Robot implements Workable {

    @Override
    public void work() {
        System.out.println("Robot working");
    }
}
```

Lợi ích

- Interface nhỏ.
- Class chỉ phụ thuộc vào thứ nó cần.
- Giảm coupling.
- Dễ implement và test.

2.5. D – Dependency Inversion Principle

Có hai ý chính:

1. High-level module không nên phụ thuộc trực tiếp vào low-level module.
2. Cả hai nên phụ thuộc vào abstraction.

Ví dụ không tốt:

```
class MySQLDatabase {
```

```
    public void save() {
    }
}

class UserService {

    private MySQLDatabase database = new MySQLDatabase();

    public void saveUser() {
        database.save();
    }
}
```

UserService bị phụ thuộc trực tiếp vào MySQLDatabase.

Nếu chuyển sang PostgreSQL phải sửa code.

Có thể tạo abstraction:

```
interface Database {

    void save();
}
```

Implementation:

```
class MySQLDatabase implements Database {

    @Override
    public void save() {
        System.out.println("Save to MySQL");
    }
}
```

Service:

```
class UserService {
```

```
    private final Database database;

    public UserService(Database database) {
        this.database = database;
    }

    public void saveUser() {
        database.save();
    }
}
```

Sử dụng:

```
Database database = new MySQLDatabase();

UserService service = new UserService(database);
```

Đây cũng chính là tư duy nền tảng của **Dependency Injection** trong Spring.

4. Interface và Abstract Class

4.1. Interface

Interface dùng để định nghĩa một **contract** mà class phải tuân theo.

Ví dụ:

```
interface Payment {

    void pay();
}
```

Class implement:

```
class MomoPayment implements Payment {

    @Override
    public void pay() {
```

```
        System.out.println("Pay with MoMo");
    }
}
```

Một class có thể implement nhiều interface:

```
class SmartPhone implements Camera, GPS, Bluetooth {
}
```

Khi nên sử dụng Interface?

Sử dụng interface khi muốn mô tả:

"Object này có khả năng/thực hiện behavior này."

Ví dụ:

Payable

Flyable

Runnable

Serializable

Comparable

4.2. Abstract Class

Abstract class phù hợp khi các class có **mối quan hệ gần nhau** và muốn chia sẻ code/state.

Ví dụ:

```
abstract class Animal {

    protected String name;

    public Animal(String name) {
        this.name = name;
    }

    public void eat() {
        System.out.println(name + " is eating");
```

```
    }

    public abstract void sound();
}
```

Class con:

```
class Dog extends Animal {

    public Dog(String name) {
        super(name);
    }

    @Override
    public void sound() {
        System.out.println("Woof");
    }
}
```

Abstract class có thể có:

- Fields.
- Constructor.
- Concrete methods.
- Abstract methods.

4.3. Interface vs Abstract Class

Interface	Abstract Class
Mô tả contract/capability	Mô tả base class
Không dùng constructor để tạo state object	Có constructor
Có thể implement nhiều interface	Java chỉ extends một class
Phù hợp cho behavior	Phù hợp chia sẻ state + behavior
Giảm coupling	Dùng khi có quan hệ gần giữa các class

Ví dụ:

```
interface Flyable {
    void fly();
}
```

thể hiện **khả năng bay**.

Trong khi:

```
abstract class Animal {
    String name;

    abstract void sound();
}
```

thể hiện một **nhóm object có chung bản chất**.

5. Static

static thuộc về **class**, không thuộc về từng object.

Ví dụ:

```
class Student {

    static String university = "HUST";

    String name;
}
```

Có thể truy cập:

```
System.out.println(Student.university);
```

Không cần:

```
Student s = new Student();
```

5.1. Static variable

```
class Counter {

    static int count = 0;
```

```
    Counter() {
        count++;
    }
}
new Counter();
new Counter();
new Counter();

System.out.println(Counter.count);
```

Kết quả:

3

count được dùng chung giữa các object.

5.2. Static method

```
class MathUtils {

    public static int add(int a, int b) {
        return a + b;
    }
}
```

Sử dụng:

```
int result = MathUtils.add(10, 20);
```

Không cần tạo object.

Lưu ý

Static method không thể truy cập trực tiếp instance field:

```
class Test {

    int value;
```

```
    static void show() {
        // System.out.println(value); // Error
    }
}
```

Vì value thuộc về object, trong khi show() thuộc về class.

6. Java Collections

Java Collections Framework cung cấp các cấu trúc dữ liệu phổ biến:

List

Set

Queue

Map

Một số implementation phổ biến:

ArrayList

LinkedList

HashSet

TreeSet

HashMap

TreeMap

ArrayDeque

ConcurrentHashMap

6.1. ArrayList

ArrayList sử dụng dynamic array bên trong.

```
List<Integer> numbers = new ArrayList<>();


numbers.add(10);
numbers.add(20);
numbers.add(30);
```

Đặc điểm:

Operation	Average
get(index)	O(1)
add cuối	O(1) amortized
add/remove đầu	O(n)
search	O(n)

Phù hợp khi:

- Đọc bằng index nhiều.
- Thêm phần tử ở cuối.
- Ít insert/remove ở giữa.

6.2. LinkedList

LinkedList sử dụng linked nodes.

Các operation ở đầu/cuối có thể rất nhanh, nhưng truy cập theo index cần traversal.

List<Integer> numbers = new LinkedList<>();

get(index) có complexity:

O(n)

Vì vậy không nên mặc định dùng LinkedList chỉ vì "insert/delete nhanh".

Trong nhiều trường hợp thực tế, ArrayList có locality tốt và hiệu năng tổng thể tốt hơn.

6.3. HashMap

HashMap lưu dữ liệu dạng:

key -> value

Ví dụ:

Map<String, Integer> scores = new HashMap<>();

scores.put("Duy", 90);

scores.put("Nam", 80);

System.out.println(scores.get("Duy"));

Average complexity:

put O(1)

get O(1)

remove O(1)

Hiệu năng phụ thuộc vào:

- hashCode()
- equals()
- Phân bố hash.
- Collision.

Khi sử dụng object làm key cần override equals() và hashCode() đúng contract.

6.4. TreeMap / TreeSet

TreeMap và TreeSet duy trì dữ liệu theo thứ tự.

Complexity thường:

get O(log n)

put O(log n)

remove O(log n)

Đổi lại có khả năng:

- Sorting.
- Range query.
- Ordered traversal.

7. Thread-safe Collections

Trong môi trường multi-thread, nhiều thread có thể cùng truy cập một collection.

Ví dụ:

List<Integer> list = new ArrayList<>();

ArrayList không đảm bảo thread safety.

Một số lựa chọn:

7.1. Collections.synchronizedList()

List<Integer> list =

    Collections.synchronizedList(new ArrayList<>());

Các operation cơ bản được đồng bộ hóa.

Tuy nhiên khi iterate vẫn cần synchronization:

```
synchronized (list) {
    for (Integer value : list) {
        System.out.println(value);
    }
}
```

7.2. CopyOnWriteArrayList

```
List<Integer> list = new CopyOnWriteArrayList<>();
```

Đặc điểm:

- Read nhiều.
- Write ít.
- Mỗi lần write có thể tạo copy của underlying array.

Phù hợp với:

Many readers

Few writers

Không phù hợp khi write rất thường xuyên.

7.3. ConcurrentHashMap

```
Map<String, Integer> map =
        new ConcurrentHashMap<>();
```

Phù hợp khi nhiều thread cùng đọc/ghi Map.

So với:

Collections.synchronizedMap(...)

ConcurrentHashMap được thiết kế cho concurrent access hiệu quả hơn, giảm mức độ contention bằng cơ chế concurrency bên trong.

8. So sánh Collections

Collection	Đặc điểm	Use case
ArrayList	Random access nhanh	Đọc nhiều
LinkedList	Linked nodes	Một số queue/deque use case

Collection	Đặc điểm	Use case
HashSet	Không đảm bảo thứ tự	Unique elements
TreeSet	Có thứ tự	Sorted unique elements
HashMap	Key-value nhanh	Lookup
TreeMap	Key-value có thứ tự	Sorted map
ConcurrentHashMap	Thread-safe map	Multi-thread
CopyOnWriteArrayList	Thread-safe read-heavy	Nhiều read, ít write

Khi lựa chọn Collection không nên chỉ dựa vào Big-O mà cần xem:

- Pattern đọc/ghi.
- Kích thước dữ liệu.
- Memory.
- Cache locality.
- Thread safety.
- Ordering requirement.

9. Exception Handling

Exception là cơ chế Java sử dụng để xử lý những tình huống bất thường xảy ra trong quá trình chạy chương trình.

Ví dụ:

```
int result = 10 / 0;
```

Có thể xảy ra:

ArithmeticException

9.1. try-catch

```
try {
    int result = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero");
}
```

Flow:

try

↓

Exception xảy ra

↓

catch

↓

continue program

9.2. finally

finally thường dùng cho cleanup.

```
try {
    // code
} catch (Exception e) {
    // handle
} finally {
    // cleanup
}
```

Tuy nhiên với resource như file/database connection, nên ưu tiên **try-with-resources**.

9.3. Try-with-resources

Ví dụ:

```
try (BufferedReader reader =
        new BufferedReader(new FileReader("data.txt"))) {


    String line = reader.readLine();


} catch (IOException e) {
    System.out.println("Cannot read file");
}
```

Resource được tự động đóng sau khi sử dụng.

Điều này an toàn hơn so với việc tự gọi:

reader.close();

9.4. Checked Exception và Unchecked Exception

Checked Exception

Compiler yêu cầu phải xử lý hoặc khai báo.

Ví dụ:

IOException

SQLException

Ví dụ:

```
public void readFile() throws IOException {
}
```

Unchecked Exception

Kế thừa từ RuntimeException.

Ví dụ:

NullPointerException

IllegalArgumentException

ArithmeticException

IndexOutOfBoundsException

Compiler không bắt buộc phải catch.

9.5. throw và throws

throw dùng để **ném một exception cụ thể**:

```
if (age < 0) {
    throw new IllegalArgumentException("Age cannot be negative");
}
```

throws dùng để **khai báo method có thể ném exception**:

```
public void readFile() throws IOException {
}
```

10. Exception Handling Best Practices

10.1. Không catch Exception một cách tùy tiện

Không nên:

```
try {
    // code
} catch (Exception e) {
}
```

Vì:

- Nuốt exception.
- Khó debug.
- Không biết lỗi gì xảy ra.

Nên catch exception cụ thể:

```
try {
    readFile();
} catch (IOException e) {
    // handle IOException
}
```

10.2. Không nuốt exception

Không nên:

```
catch (Exception e) {
}
```

Nếu không xử lý được thì ít nhất cần log hoặc propagate exception phù hợp.

10.3. Exception message phải có ý nghĩa

Không nên:

```
throw new RuntimeException("Error");
```

Nên:

```
throw new IllegalArgumentException(
    "Age must be greater than or equal to 0"
);
```

Thông tin lỗi càng rõ thì debugging càng dễ.

10.4. Không dùng Exception để điều khiển flow bình thường

Không nên:

```
try {
    Integer.parseInt(input);
} catch (NumberFormatException e) {
    // normal flow
}
```

Nếu có thể validate trước thì nên validate.

Exception nên đại diện cho **exceptional situation**, không phải logic bình thường của chương trình.

10.5. Không catch rồi throw lại một cách vô nghĩa

Không nên:

```
try {
    service.save();
} catch (Exception e) {
    throw e;
}
```

Nếu không thêm context hoặc xử lý gì thì việc catch này không mang lại giá trị.

11. Thực hành tổng hợp

Tôi thực hành kết hợp các kiến thức trên bằng một ví dụ đơn giản về hệ thống thanh toán.

Interface

```
interface Payment {

    void pay(double amount);
}
```

Các implementation

```
class MomoPayment implements Payment {

    @Override
    public void pay(double amount) {
```

```
        System.out.println("Pay " + amount + " using MoMo");
    }
}
class VisaPayment implements Payment {

    @Override
    public void pay(double amount) {
        System.out.println("Pay " + amount + " using Visa");
    }
}
```

Service

```
class PaymentService {

    private final Payment payment;

    public PaymentService(Payment payment) {
        this.payment = payment;
    }

    public void process(double amount) {

        if (amount <= 0) {
            throw new IllegalArgumentException(
                "Amount must be greater than 0"
            );
        }

        payment.pay(amount);
    }
```

```
}
```

Main

```
public class Main {

    public static void main(String[] args) {

        Payment payment = new MomoPayment();

        PaymentService service =
                new PaymentService(payment);

        service.process(100000);
    }
}
```

Ví dụ trên áp dụng nhiều kiến thức:

Interface

↓

Polymorphism

↓

Dependency Inversion

↓

Dependency Injection

↓

Exception Handling

↓

Encapsulation

Nếu thêm:

```
class PaypalPayment implements Payment {
    @Override
```

```
    public void pay(double amount) {
        System.out.println("Pay using Paypal");
    }
}
```

thì PaymentService không cần sửa.

Điều này thể hiện rõ **Open/Closed Principle**.